Learning Strategies for Grasping and Tossing Rigid Objects to Fixed Targets with Robotic Manipulators

Anonymous Authors

***Abstract*— Tossing extends the reach of a manipulator and speeds up pick-and-place operations, but its outcome depends on both the selected grasp and the release conditions. In this work, we generate dense grasp labels automatically in simulation, using a geometric criterion. We train a network to infer grasps from color and depth (RGB-D) images, and then refine its policy through simulated tosses to favor the grasps that improve the subsequent task. We combine the selected grasp with a ballistic model and a trajectory planner to generate the tossing motion. We evaluate the policy on the physical robot without fine-tuning it on real data. The system grasped 97.56% of the seen parts and 96.15% of the unseen ones, with a full-task success rate of 85.12% and 73.56%, respectively.**

***Index Terms*— Deep Learning in Grasping and Manipulation, Grasping, RGB-D Perception, Dynamic Manipulation, Robot Tossing**

I. Introduction

TOSSING extends the delivery range of a manipulator beyond its kinematic workspace by allowing an object to complete part of the transfer in free flight. The robot only needs to reach a suitable release state, allowing targets to be placed outside the region accessible to the gripper [1]. For repeated transfer operations in manufacturing and sorting, releasing the object before reaching its destination can also shorten the transport motion and allow the robot to begin its return while the object is still in flight. These reductions can shorten cycle times and increase throughput [2], [3].

Selecting a grasp for tossing requires considering both whether the robot can acquire the object and how the object will move during acceleration and release. This makes grasp selection part of the tossing problem itself, following the principle of task-oriented grasping: evaluating a grasp according to its contribution to the manipulation that follows acquisition [4]. For tossing, that contribution extends to the object's flight, with success determined by whether it reaches the assigned target.

Evaluating grasp candidates through complete grasp-and-toss attempts requires executing each action before obtaining feedback about its suitability. Geometric labeling can provide supervision for grasp acquisition without those executions, assessing candidate positions and orientations according to the observed geometry and the gripper's constraints [5], [6]. These labels assess whether the object can be acquired, whereas grasp-and-toss outcomes provide information about whether the selected grasp contributes to reaching the target [7], [8]. Combining these sources allows the policy to learn grasp feasibility before using complete task outcomes to distinguish grasps that favor the subsequent toss.

In this work, we first train the policy to identify geometrically feasible grasps because the short stroke of our parallel-jaw gripper leaves only a few millimeters of clearance around the parts. Errors of a few pixels in the observed heightmap can prevent acquisition altogether, reducing the opportunities to obtain feedback from an actual toss. Geometric pretraining gives refinement an initial policy that can acquire the parts, allowing subsequent experience to inform grasp selection through the tossing outcome.

As shown in Fig. 1, the first training stage supervises the entire action grid through gripper opening and collision checks, while the second evaluates the loss only at the executed grasp. Both stages run in simulation, and a ballistic model and trajectory planner generate the tossing motion from the selected grasp and target. The refined policy achieves 94.1% full-task success in simulation. Deployed on the physical robot without fine-tuning on real data, it achieves 96.15% grasp success and 73.56% full-task success on previously unseen parts.

II. Related Work

Robotic tossing. Accurate tossing requires producing a suitable release state and accounting for how the robot's motion is transmitted to the object. Model-based planning connects desired landing conditions to executable robot trajectories [2], while uncertainty during gripper opening and contact at release has motivated robust motion design, explicit contact models, and proprioceptive flight prediction [9], [10], [11]. These approaches address effects that a ballistic description of free flight alone does not capture. Learning provides another means of adapting the tossing motion from simulated or physical experience, including adaptation with few physical samples, correction of landing position and orientation, and tactile adjustment for deformable objects [12], [13], [14]. At the system level, acquisition and delivery have been coordinated through specialized scoop-and-flick mechanisms and bimanual manipulation [15], [3]. Integrating additional actions also allows a robot to modify the pickup scene before attempting acquisition, as in the pushing, grasping, and tossing system of Kasaei and Kasaei [16]. Its components are trained separately, however, so the final tossing outcome does not refine grasp selection.

Grasping for a subsequent task. Task-oriented grasping incorporates requirements beyond acquiring and holding an object, using downstream task execution or category-level correspondences to learn suitable grasps [4], [17]. Task

Author names, affiliations and funding acknowledgments are omitted for double-anonymous review.

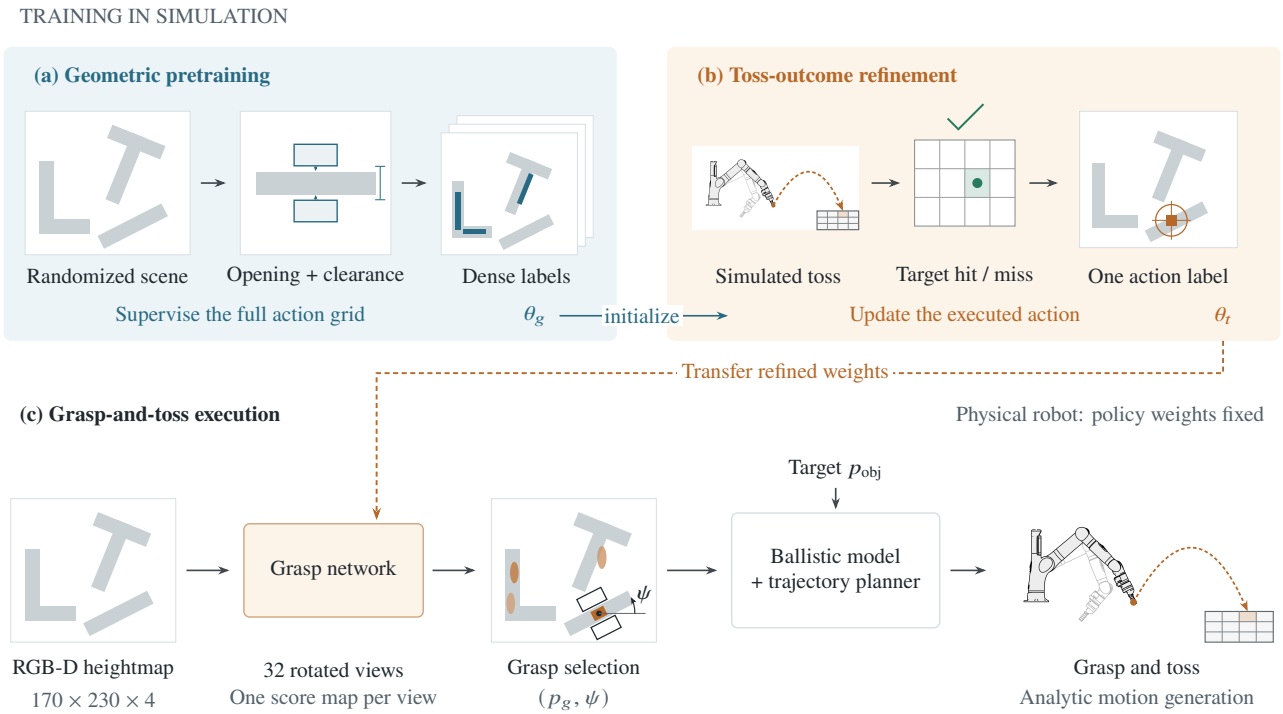


Fig. 1. Grasp-and-toss system. (a) Dense geometric pretraining produces weights θ_g. (b) Simulated task outcomes refine them to θ_t. (c) The policy selects a grasp, and a ballistic model and trajectory planner generate the toss. Weights remain fixed during physical deployment. Scenes, labels, score maps, and flights are schematic.

information can also come from human-activity data or semantic knowledge supplied by language and foundation models [18], [19], [20]. For tossing, grasp suitability extends to the object's flight after contact ends. TossingBot and the collaborative grasp–toss method of Hou et al. use tossing outcomes to supervise grasp selection while learning corrections to a physics-based toss [7], [8]. These methods provide direct precedents for our outcome-based refinement; our approach initializes the grasp policy with geometric supervision and trains it entirely in simulation.

Automatic supervision for grasp learning. Analytical grasp evaluation and spatial grasp prediction allow many candidates to be assessed within a scene [21], [22], [23], [24]. The coverage of geometric supervision ranges from systematic evaluation of image positions and orientations [5] to spatial label maps constructed from geometric proposals [6], [25]. Executed grasps provide outcome-based supervision, with physical experience and equivariant models supporting visual grasp learning [26], [27]. Hou and Li combine these training phases by pretraining on coarse maps derived from manually annotated object centers and then learning from grasping outcomes [28]. Our method instead initializes the policy with geometric feasibility labels throughout the position–orientation action grid, then refines it using the outcome of the complete grasp-and-toss task.

III. Methodology

A. Problem Formulation

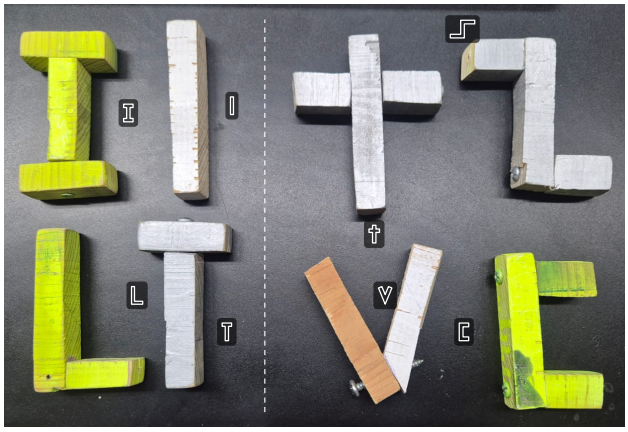
We consider a pick-and-toss task in which a manipulator with a parallel-jaw gripper transfers rigid parts from a tabletop to a fixed target. Each attempt begins with a color and depth (RGB-D) observation of the pickup area and a target position p_{obj}. The robot must acquire a part, lift it, and release it during a tossing motion so that it reaches the assigned target. The parts used in our evaluation are shown in Fig. 2.

As in TossingBot [7], we formulate pick-and-toss as two parameterized motion primitives: a vertical grasp specified by its position and orientation, followed by a toss specified by the release position and velocity. The grasp policy selects $\phi_g = (p_g, \psi)$ from an orthographic scene representation I, where p_g is the grasp position and ψ is the gripper rotation about the vertical axis. After a successful grasp, a ballistic model and a trajectory planner determine the tossing motion for the assigned target, including the release position r and velocity v. The grasp policy receives the scene observation; the target position is used by the tossing primitive.

An attempt succeeds when the robot both acquires the part and tosses it to the assigned target; failed grasps and tosses that miss that target are full-task failures.

B. Observation

The grasp policy uses an orthographic RGB-D representation of the pickup area, in which each pixel corresponds



(a) Seen parts (b) Unseen parts

Fig. 2. The eight physical parts used in evaluation: (a) shapes represented in training and (b) previously unseen shapes. All parts are 19 mm thick, with segment widths of 14–18 mm. The photograph is cropped and rotated for display.

to a fixed physical footprint so that predicted grasp locations can be mapped directly to positions on the tabletop.

To construct this representation, the camera point cloud is transformed into the robot reference frame, cropped to the pickup area, and projected onto a grid aligned with the tabletop. The grid covers 0.34×0.46 m at 2 mm per pixel, yielding an observation $I \in \mathbb{R}^{170\times230\times4}$. Before inference, each channel is normalized using its mean and standard deviation precomputed from 500 domain-randomized simulated observations.

To approximate the measurements of the physical Xtion sensor, we simulate depth-dependent noise, quantization, and missing depth measurements. During training, we apply visual domain randomization [29] by varying part and background colors, lighting, camera pose, and RGB brightness, contrast, and per-channel gains. Camera translation and rotation perturbations are bounded per axis by 3 mm and $0.3°$, respectively, while physical parameters are not randomized.

C. Grasp Network

We use a fully convolutional U-Net with three downsampling stages, channel widths of 32, 64, 128, and 256, and bilinear upsampling. The network uses two 3×3 convolutions per block, each followed by group normalization and a rectified linear unit (ReLU) activation, and ends with a 1×1 convolution that produces grasp-score logits at the input resolution.

Following the rotation-based grasp representation used in TossingBot [7], we pad the observation to preserve the pickup area before evaluating the same network on 32 rotated views, covering $360°$ in steps of $11.25°$.

The resulting scores Q_{ouv} index orientation o and image position (u, v). During evaluation, we select the highest-scoring candidate across all positions and orientations, using its pixel location to determine p_g and its rotation index to determine ψ.

D. Toss Primitive and Motion Generation

For each target, the tossing primitive computes a nominal release state using a ballistic point-mass model. Let d_h denote the horizontal distance from the nominal release position $\hat{r}$ to the target, and d_z the target height relative to $\hat{r}$. For a prescribed launch angle α above the horizontal, the required release speed is

$$\|\hat{v}\| = \sqrt{\frac{g d_h^2}{2\cos^2\alpha\,(d_h\tan\alpha - d_z)}}, \tag{1}$$

where g is gravitational acceleration. The model neglects aerodynamic forces and treats the object as a point released from the end effector.

To generate the motion, the base joint is oriented toward the target, accounting for the lateral offset of the gripper. Joints θ_1, θ_3, and θ_5 then produce a planar tossing motion, while the remaining joints stay fixed. We optimize the joint trajectory and its duration subject to joint position, velocity, acceleration, and torque limits, together with workspace bounds. The release direction and speed are constrained, while the objective penalizes deviation from $\hat{r}$, motion duration, acceleration, and jerk. Robot kinematics and inverse dynamics are evaluated with Pinocchio [30]. The optimization is implemented in CasADi [31] and solved with IPOPT [32].

The robot moves to the computed starting configuration and executes the trajectory with gripper opening synchronized to the planned release time.

E. Stage 1: Dense Geometric Labels

We initialize the grasp policy with dense supervision computed directly from scene geometry in MuJoCo [33]. Each position and orientation in the action grid receives a binary label y_i, indicating whether the candidate satisfies the geometric grasp criterion, providing supervision across the scene without executing individual grasps.

For a candidate centered on a part, the grasp height is set halfway between the tabletop and the local part surface. A positive label requires that the local cross-section fit within the gripper's 12–24 mm opening range and that both fingers descend to the grasp height without intersecting any part; all other candidates receive negative labels. For collision checking, each finger is modeled by its nominal 30×15 mm horizontal footprint, expanded by 1 mm per side to provide a positioning margin. These checks evaluate finger clearance and opening compatibility without simulating closure or lifting.

Since feasible grasps occupy a small fraction of the action grid, we average the positive and negative losses separately. For each orientation map, the training loss is

$$\mathcal{L}_g = \frac{1}{2}\left(\frac{1}{N^+}\sum_{i:y_i=1}\ell_i + \frac{1}{N^-}\sum_{i:y_i=0}\ell_i\right), \tag{2}$$

where ℓ_i is the binary cross-entropy between the predicted logit and its label, and N^+ and N^- are the numbers of

positive and negative labels. If a map contains only one class, its mean loss is used.

The dataset contains 1,000 scenes, each with one to ten parts sampled from 60 variants with different segment lengths and cross-sectional widths. We render and store the observations with domain randomization, using 850 scenes for training and 150 for validation. The network is trained from random initialization for ten epochs using stochastic gradient descent (SGD), with a learning rate of 10^{-4} and a batch size of four.

F. Stage 2: Refinement from the Toss Outcome

We continue training the grasp policy obtained in Stage 1 using simulated grasp-and-toss episodes, applying the same randomization settings at each observation capture. Following the use of tossing outcomes to supervise grasp selection in TossingBot [7], the task label y_i^t is positive when the part reaches the assigned target and negative otherwise, including failed grasps. Target positions are sampled uniformly within the training region, exposing the policy to different tossing distances and lateral offsets.

The task loss is evaluated only at the logit corresponding to the executed grasp position and orientation:

$$\mathcal{L}_{t,i} = w_i \, \ell \left(Q_{o_i u_i v_i}, y_i^t \right), \tag{3}$$

where ℓ denotes binary cross-entropy with logits. We set $w_i = 0.5$ when acquisition succeeds but the toss misses its target, and $w_i = 1$ otherwise. We downweight these cases because a missed toss may reflect release or motion errors as well as grasp selection. Each episode is stored in a prioritized replay buffer, and the network is updated after each episode using SGD with a learning rate of 10^{-5} and a batch size of 16.

During refinement, the policy usually selects the highest-scoring action, with exploratory actions sampled from the geometrically feasible candidates using a Boltzmann distribution over their predicted logits. The exploration probability and sampling temperature follow a 3,000-episode linear annealing schedule, decreasing from 0.1 to 0.02 and from 0.5 to 0.15, respectively.

We refine the policy on 39 of these part variants and select the checkpoint at episode 2,500 based on simulation performance, keeping its parameters fixed throughout physical evaluation.

IV. Experiments

A. Experimental Setup

We evaluate the policy in simulation and on the physical setup shown in Fig. 3, verifying physical tossing outcomes manually. The seven-degree-of-freedom Sawyer manipulator has a maximum reach of 1.26 m and is equipped with a ClickSmart Pneumatic Small Gripper. An ASUS Xtion Pro RGB-D camera observes the pickup area, while a separate overhead camera is used to localize the targets and estimate their distances relative to the robot.

Both evaluations use the eight parts shown in Fig. 2: four with shapes represented during training and four with previously unseen shapes. We fabricated one physical instance of each shape, all 19 mm thick with segment widths of 14–18 mm.

The 12 targets form a three-row, four-column array, with rows located 1.56, 1.72, and 1.90 m from the robot base, beyond the manipulator's reach. Each bin has a 7.5×15 cm opening and an 8 cm depth.

In our Sawyer setup, trajectory tracking errors caused the release speed to fall below the planned value. We compensated for this discrepancy using an affine speed correction calibrated on the physical robot at the three target-row distances and kept the correction fixed across all parts and targets during evaluation.

During physical evaluation, all eight parts are placed together in the pickup area and replenished after the table is cleared. We visit all 12 targets in randomly ordered cycles and continue until 100 tosses have been executed for each part, retaining failed grasp attempts in the record.

We report grasp success per attempt, tossing success per executed toss, and full-task success per attempt. Physical tossing success is determined by entry into the assigned target bin, including entry after a wall rebound or followed by a bounce out. A toss that first enters a different bin is counted as a failure.

B. Simulation Results

We compare the U-Net trained only with geometric labels against three variants refined using tossing outcomes: U-Net with 32 orientations, U-Net with 16 orientations, and a residual fully convolutional network (FCN) based on the grasp architecture of TossingBot [7], with 32 orientations. The refined models are evaluated after 2,500 episodes.

Evaluation uses the eight parts and the target array described above. Each model is evaluated over 1,720 grasp attempts, selecting the highest-scoring action without exploration, with aggregate success rates reported in Table I.

The initial U-Net achieves 96.4% grasp success and 66.8% full-task success. The refined U-Net with 32 orientations achieves 98.1% and 94.1%, respectively, obtaining the highest success rates among the evaluated variants. We therefore

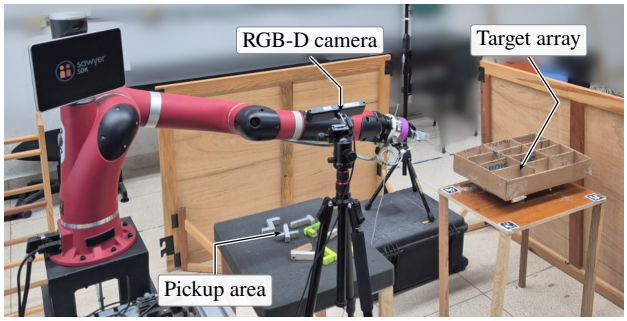


Fig. 3. Physical setup with the Sawyer manipulator, pickup area, RGB-D camera, and target array; the photograph is cropped and the background blurred for clarity.

select the refined U-Net with 32 orientations for physical evaluation.

C. Real-World Results

Table II summarizes the physical evaluation of seen and unseen parts, in which 800 of the 826 attempts proceed to tossing and 655 reach the assigned target.

Grasp success remains above 96% in both groups, with 145 of the 171 full-task failures occurring after acquisition.

The unseen and parts illustrate the variation across individual parts: their grasp success rates are 100% and 94.3%, while their tossing success rates are 55% and 66%, respectively. The other two unseen parts achieve tossing success rates of 90% and 95%.

For seen parts, tossing success decreases from 92.4% at the nearest target row to 79.5% at the farthest. Unseen parts achieve their highest row-wise success rate, 85.4%, at the intermediate row.

D. Grasp Policy Analysis

We compare the grasp maps predicted by the Stage 1 and refined networks on 256 observations recorded during physical evaluation, using the maximum grasp probability across orientations at each pixel.

We measure spatial concentration as the fraction of pixels whose score exceeds half the maximum within the recorded mask of the selected part, using the same mask for both networks. The threshold is computed separately for each network to account for their different score ranges.

The mean fraction decreases from 39.35% for the Stage 1 network to 6.92% after refinement, with a smaller fraction produced by the refined network in all 256 observations. Representative maps are shown in Fig. 4: high scores cover broader regions before refinement and become more localized afterward. This change is consistent with the transition from geometric acquisition labels to supervision based on complete grasp-and-toss outcomes.

Refinement may favor contact regions that hold the part more consistently during tossing, such as the middle of longer segments rather than short caps or junctions with limited finger clearance. Such choices could reduce sensitivity to positioning errors and part motion during acceleration and release.

TABLE I

SUCCESS RATES IN SIMULATION (%).

Model	Grasp	Toss	Full task
U-Net 32, Stage 1	96.4	69.3	66.8
U-Net 16, refined	91.8	86.1	79.1
Residual FCN 32, refined	93.4	93.5	87.3
U-Net 32, refined	**98.1**	**95.9**	**94.1**

TABLE II

PHYSICAL EVALUATION: SUCCESSES/ATTEMPTS (SUCCESS RATE).

Metric	Seen parts	Unseen parts
Grasp	400/410 (97.56%)	400/416 (96.15%)
Toss	349/400 (87.25%)	306/400 (76.50%)
Full task	349/410 (85.12%)	306/416 (73.56%)

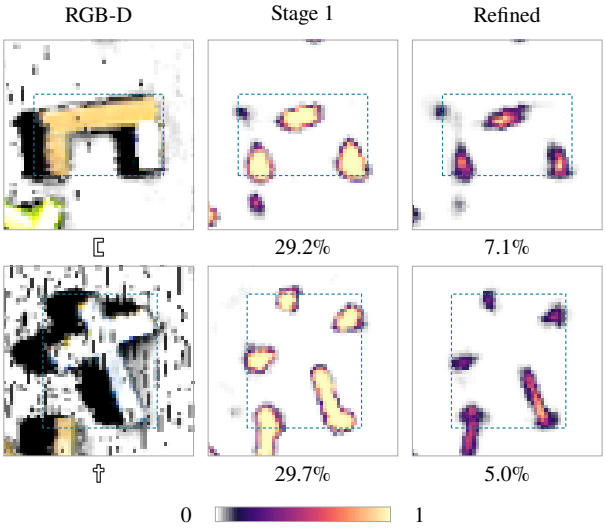


Fig. 4. Grasp score maps before and after refinement for regions cropped from two physical observations. Each row uses the same crop; dashed boxes identify the selected part. Scores are normalized independently for display. Percentages give the fraction of pixels within the part mask exceeding half its maximum score, computed separately for each network.

E. Comparison with Prior Work

Table III places our physical results alongside the residual-physics version of TossingBot [7, Table III] and the collaborative grasp-throw learning method (CGTL) of Hou et al. [8, Tables I and II]. Because the values come from independent evaluations with different manipulators, grippers, object populations, and target geometries, the reported rates do not isolate the effect of the learning method.

Our grasp success is higher in both groups, while tossing success is higher for seen parts and lower for unseen parts.

In their physical implementations, both reference methods learn grasp selection and tossing compensation through robot trials. Hou et al. additionally use object mass measurements and a learned air-resistance estimate. Our grasp policy is trained entirely in simulation and combined with a ballistic tossing primitive whose global speed correction is calibrated on the physical robot.

V. CONCLUSION

We presented a two-stage method for learning grasp selection for robotic tossing. Geometric labels provide dense initial supervision, and simulated tossing outcomes refine which grasps the policy favors. Both stages run in simulation, and the resulting grasp policy is deployed without fine-tuning on real data.

TABLE III

PHYSICAL SUCCESS RATES (%) FROM INDEPENDENT EVALUATIONS OF TOSSINGBOT, CGTL, AND OUR SYSTEM.

	Grasp		Toss	
Method	Seen	Unseen	Seen	Unseen
TossingBot	86.9	73.2	84.7	82.3
Hou et al. (CGTL)	85.0	83.3	84.3	80.0
This work	97.56	96.15	87.25	76.50

In simulation, the refined policy achieves 94.1% full-task success, compared with 66.8% for the policy trained only with geometric labels. On the physical robot, it achieves 96.15% grasp success and 73.56% full-task success on previously unseen parts, using a globally calibrated tossing primitive. Most physical task failures occur after acquisition, indicating that reliable tossing remains the main challenge in this setup.

The evaluation is limited to rectilinear parts, one physical instance per shape, and a prescribed tossing motion. Further work should test a broader range of part shapes and evaluate whether grasp-dependent release corrections improve physical tossing performance.

Physical success measures entry into the assigned target bin, including cases followed by a bounce out. Reliable sorting also requires retention; deeper bins or impact-absorbing liners could reduce such exits, but were not evaluated here.

ACKNOWLEDGMENT

The authors used Claude Code and Codex to assist with manuscript and figure preparation and software development. The authors take full responsibility for the final content and results.

REFERENCES

[1] K. M. Lynch and M. T. Mason, "Dynamic nonprehensile manipulation: Controllability, planning, and experiments," *Int. J. Robot. Res.*, vol. 18, no. 1, pp. 64–92, 1999.

[2] A. Zermane, N. Dehio, and A. Kheddar, "Planning impact-driven logistic tasks," *IEEE Robot. Autom. Lett.*, vol. 9, no. 3, pp. 2184–2191, 2024.

[3] M. Bombile and A. Billard, "Bimanual dynamic grabbing and tossing of objects onto a moving target," *Robot. Auton. Syst.*, vol. 167, p. 104481, 2023.

[4] K. Fang, Y. Zhu, A. Garg, A. Kurenkov, V. Mehta, L. Fei-Fei, and S. Savarese, "Learning task-oriented grasping for tool manipulation from simulated self-supervision," *Int. J. Robot. Res.*, vol. 39, no. 2–3, pp. 202–216, 2020.

[5] D. Wang, F. Chang, C. Liu, H. Huan, N. Li, and R. Yang, "On-policy and pixel-level grasping across the gap between simulation and reality," *IEEE Trans. Ind. Electron.*, vol. 71, no. 7, pp. 7388–7399, 2024.

[6] A. Sabzejou, F. Azimmohseni, M. Tale Masouleh, and A. Kalhor, "Skeleton-guided geometry-aware auto-labeling for robotic grasping: From 4-DoF learning to 6-DoF execution," *Int. J. Robot. Res.*, 2026, onlineFirst.

[7] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, "TossingBot: Learning to throw arbitrary objects with residual physics," *IEEE Trans. Robot.*, vol. 36, no. 4, pp. 1307–1319, 2020.

[8] Y. Hou, Z. Fang, and J. Li, "Self-supervised multi-modal learning for collaborative robotic grasp-throw," *IEEE Robot. Autom. Lett.*, vol. 9, no. 5, pp. 4250–4256, 2024.

[9] Y. Liu and A. Billard, "Tube acceleration: Robust dexterous throwing against release uncertainty," *IEEE Trans. Robot.*, vol. 40, pp. 2831–2849, 2024.

[10] ——, "On transient release dynamics in robot throwing: A sliding pivot model," *IEEE Trans. Robot.*, vol. 42, pp. 132–151, 2026.

[11] L. Chen, W. Lu, K. Zhang, Y. Zhang, L. Zhao, and Y. Zheng, "TossNet: Learning to accurately measure and predict robot throwing of arbitrary objects in real time with proprioceptive sensing," *IEEE Trans. Robot.*, vol. 40, pp. 3232–3251, 2024.

[12] M. Monastirsky, O. Azulay, and A. Sintov, "Learning to throw with a handful of samples using decision transformers," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 576–583, 2023.

[13] Y. Liu, B. Da Costa, and A. Billard, "Learning to throw-flip," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2025, pp. 3209–3216.

[14] S. Aslam, K. Kumar, P. Zhou, H. Yu, M. Y. Wang, and Y. She, "DartBot: Overhand throwing of deformable objects with tactile sensing and reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 13 644–13 661, 2025.

[15] J. Lee, S. Oh, and J. Seo, "Manipulation models for robotic high-arc object transfer and their implementation," *Appl. Sci.*, vol. 16, no. 7, p. 3205, 2026.

[16] H. Kasaei and M. Kasaei, "Harnessing the synergy between pushing, grasping, and throwing to enhance object manipulation in cluttered scenarios," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024, pp. 13 855–13 861.

[17] B. Wen, W. Lian, K. Bekris, and S. Schaal, "CaTGrasp: Learning category-level task-relevant grasping in clutter from simulation," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2022.

[18] M. Kokic, D. Kragic, and J. Bohg, "Learning task-oriented grasping from human activity datasets," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3352–3359, 2020.

[19] C. Tang, D. Huang, W. Ge, W. Liu, and H. Zhang, "GraspGPT: Leveraging semantic knowledge from a large language model for task-oriented grasping," *IEEE Robot. Autom. Lett.*, vol. 8, no. 11, pp. 7551–7558, 2023.

[20] C. Tang, D. Huang, W. Dong, R. Xu, and H. Zhang, "FoundationGrasp: Generalizable task-oriented grasping with foundation models," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 12 418–12 435, 2025.

[21] Y. Domae, H. Okuda, Y. Taguchi, K. Sumi, and T. Hirai, "Fast graspability evaluation on single depth maps for bin picking with general grippers," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2014, pp. 1997–2004.

[22] J. Mahler *et al.*, "Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics," in *Proc. Robotics: Science and Systems (RSS)*, 2017.

[23] V. Satish, J. Mahler, and K. Goldberg, "On-policy dataset synthesis for learning robot grasping policies using fully convolutional deep networks," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1357–1364, 2019.

[24] D. Morrison, P. Corke, and J. Leitner, "Learning robust, real-time, reactive robotic grasping," *Int. J. Robot. Res.*, vol. 39, no. 2–3, pp. 183–201, 2020.

[25] L. Zhang, K. Bai, Z. Chen, Y. Shi, and J. Zhang, "Towards precise model-free robotic grasping with sim-to-real transfer learning," in *Proc. IEEE Int. Conf. Robotics and Biomimetics (ROBIO)*, 2022, pp. 1–8.

[26] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen, "Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection," *Int. J. Robot. Res.*, vol. 37, no. 4–5, pp. 421–436, 2018.

[27] X. Zhu, D. Wang, G. Su, O. Biza, R. Walters, and R. Platt, "On robot grasp learning using equivariant models," *Auton. Robots*, vol. 47, no. 8, pp. 1175–1193, 2023.

[28] Y. Hou and J. Li, "Accelerating grasp learning via pretraining with coarse affordance maps of objects," arXiv:2205.05508, 2022.

[29] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.

[30] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiraux, O. Stasse, and N. Mansard, "The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," in *Proc. IEEE/SICE Int. Symp. System Integration (SII)*, 2019, pp. 614–619.

[31] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: A software framework for nonlinear optimization and optimal control," *Math. Program. Comput.*, vol. 11, no. 1, pp. 1–36, 2019.

[32] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Math. Program.*, vol. 106, no. 1, pp. 25–57, 2006.

[33] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.